

程序设计实习大作业报告

项目名称：卷王博雅塔

一、程序简介

“卷王博雅塔”使用 **C++17 与 Qt Widgets** 开发，是一个包含地图、卡牌战斗、随机事件、商店和存档等模块的桌面游戏。程序的主要开发难点集中在多页面管理、局内状态维护、回合制战斗、卡牌拖拽、随机地图生成、数据驱动事件和 JSON 存档等方面

玩家在随机生成的地图中逐层前进，通过战斗、事件、商店、奖励和宿舍等节点构筑卡组、收集加成并维持“心情”，最终挑战“期末考试” Boss

主要功能如下：

- 随机地图**：共 19 层，每层包含战斗、事件、商店、奖励、宿舍等节点，最后一层为期末考试
- 卡牌战斗**：支持抽牌、耗费精力、攻击、防御、回复、抽牌、虚弱、易伤、效率值等效果
- 卡组构筑**：战斗后可从随机卡牌中选择一张加入卡组，也可在宿舍或商店升级、删除卡牌
- 随机事件**：事件包含多页故事、多个选项和复合效果
- 商店与奖励**：可购买卡牌、燕园加成、饮料，也可使用删牌服务
- 宿舍系统**：可选择回复心情、升级卡牌或删除卡牌
- 成绩结算**：根据战斗表现获得平时分，Boss 战后计算期末成绩、总评和绩点
- 存档与音频**：支持继续游戏、自动存档、场景音乐切换和音量设置

二、程序总体结构

源码主要分为 **core** 和 **ui** 两部分：

```
1 Spire/
2  └─ main.cpp
3  └─ mainwindow.h/.cpp
4  └─ core/
5     └─ GameState.h
6     └─ GameRandom.h
7     └─ GameBalance.h
8     └─ GameText.h
9     └─ Card.h
10    └─ CardLibrary.h
```

```
11 | | └─ Hand.h
12 | | └─ Enemy.h
13 | | └─ EventLibrary.h
14 | | └─ Relic.h
15 | | └─ Potion.h
16 | | └─ AudioManager.h
17 | └─ ui/
18 |   └─ MapWidget.h
19 |   └─ BattleWidget.h/.cpp
20 |   └─ EventWidget.h
21 |   └─ ShopWidget.h
22 |   └─ RewardWidget.h
23 |   └─ RestWidget.h
```

`core` 目录保存与具体界面相对独立的数据和规则，`ui` 目录负责显示和交互。`MainWindow` 位于两者之间，负责创建页面、切换页面并协调各模块

三、主要模块与代码分析

3.1 程序入口、页面调度与对象管理

`main.cpp` 负责创建 `QApplication` 和 `MainWindow`，程序主体逻辑集中在主窗口及各功能模块中

`MainWindow` 继承自 `QMainWindow`，内部使用 `QStackedWidget` 管理地图、战斗、事件、商店、奖励、宿舍和结算等页面。页面通过 `setCurrentWidget()` 切换，使不同功能共用同一个顶层窗口，窗口尺寸、设置按钮和音频控制也能统一维护

部分页面需要动态创建。例如每场战斗都具有独立的敌人、手牌、动画和定时器，因此战斗结束后，程序会将旧页面从页面栈中移除，再调用 `deleteLater()` 延迟销毁。这样可以避免上一场战斗的状态残留，也能防止正在事件循环中的对象被立即删除。

各子页面通过回调函数向主窗口报告结果：

```
1 std::function<void(bool)> m_finishCallback;
2 std::function<void(MapNodeType)> m_nodeHandler;
3 std::function<void()> m_finishHandler;
```

例如，战斗页面只返回胜负，地图页面只返回节点类型，具体的页面跳转和存档操作仍由主窗口统一处理。该方式减少了子页面对 `MainWindow` 的直接依赖

3.2 全局状态、随机数与参数配置

局内长期数据由 `GameState` 保存。该类采用单例模式：

```
1 static GameState &instance()
2 {
3     static GameState state;
4     return state;
5 }
```

地图、战斗、事件和商店可以访问同一份状态。保存的数据包括当前数值、楼层、卡组、加成、饮料、战斗统计和下一场战斗的临时效果等

状态类通过 `heal()`、`loseHp()`、`addCoins()`、`spendCoins()`、`addCard()` 等接口修改数据，并使用 `qBound`、`qMax` 统一处理边界。例如：

```
1 void setHp(int hp)
2 {
3     m_hp = qBound(0, hp, m_maxHp);
4 }
```

这样可以避免各个界面重复编写数值检查，也能保证状态修改规则一致

`GameRandom` 对 `QRandomGenerator` 进行封装，地图生成、敌人组合、奖励和洗牌都使用同一个随机数源。玩家可以输入固定种子，从而重复生成相同的初始随机内容，便于调试和复现问题

项目将主要数值集中放在 `GameBalance.h`，将界面文字与资源路径集中放在 `GameText.h`。数值、文本和业务逻辑相互分离，后期调整卡牌、敌人或地图参数时，无需修改界面代码

3.3 卡牌、牌堆与敌人设计

`Card` 类用于保存一张卡牌的规则数据，包括稳定 ID、费用、伤害、防御、抽牌、状态效果和升级标记等。`CardLibrary` 负责根据 ID 创建卡牌，并从 `GameBalance` 中读取对应数值

存档时只记录卡牌 ID 和升级状态，读档时再由 `CardLibrary` 重新构造完整对象。这种方式减少了 JSON 中的重复数据，也便于统一修改卡牌描述和数值

`Hand` 类维护三组数据：

```
1 QList<Card> m_drawPile;
2 QList<Card> m_discardPile;
3 QList<Card> m_cards;
```

战斗开始时复制卡组并洗牌；抽牌堆为空时，将弃牌堆重新洗回抽牌堆。洗牌采用 Fisher-Yates 算法：

```
1 for (int i = cards.size() - 1; i > 0; --i) {
2     const int j = GameRandom::instance().bounded(i + 1);
3     qSwap(cards[i], cards[j]);
4 }
```

该算法只需一次线性遍历，并能保证每种排列等概率出现

`Enemy` 类保存敌人的数值、状态和行动序列。敌人行为由 `EnemyAction` 描述，支持攻击、回复、强化、格挡和组合行动。不同敌人通过工厂函数配置数据，战斗时循环读取行动列表，因此无需为每种敌人建立独立派生类

3.4 战斗流程与回合状态机

`BattleWidget` 负责战斗初始化、回合切换、卡牌结算、敌人行动、动画和胜负判断。其主要状态包括：

```
1 Hand m_hand;
2 QList<Enemy> m_enemies;
3 int m_playerHp;
4 int m_playerBlock;
5 int m_playerStrength;
6 int m_energy;
7 int m_turnNumber;
8 bool m_playerTurn;
9 bool m_battleEnded;
```

玩家回合开始时，程序恢复精力、抽取手牌并刷新状态。点击结束回合后，程序使用 `m_enemyTurnIndex` 按顺序执行敌人行动，所有敌人完成后再开始下一玩家回合

整体过程为：

```
1 开始玩家回合
2     ↓
3 恢复精力并抽牌
4     ↓
5 玩家出牌
6     ↓
7 结束回合
8     ↓
9 依次执行敌人行动
10    ↓
11 检查胜负
12    ↓
13 开始下一回合
```

出牌由 `playCard()` 统一处理。程序先检查当前回合、卡牌下标、费用和目标是否合法，再结算伤害、防御、回复、抽牌、虚弱、易伤和效率值等效果。卡牌完成结算后进入弃牌堆，带有消耗属性的卡牌则进入消耗堆

由于动画和敌人行动通过定时器异步执行，代码使用 `m_playerTurn` 和 `m_battleEnded` 限制重复输入，并在延迟回调中再次检查战斗状态，避免战斗已经结束后继续结算

3.5 卡牌拖拽、自定义布局与动画

战斗界面重写 `eventFilter()`，统一监听卡牌和饮料控件的鼠标事件

鼠标按下时记录位置，移动距离超过 Qt 的拖拽阈值后，卡牌暂时脱离手牌布局并跟随鼠标。释放时，程序通过全局坐标判断卡牌是否落入某个敌人控件区域：

```
1  按下卡牌
2    ↓
3  移动超过拖拽阈值
4    ↓
5  卡牌跟随鼠标
6    ↓
7  判断释放位置
8    └─ 有效目标：执行效果
9    └─ 无效目标：返回手牌
```

手牌由 `layoutHandButtons()` 动态排列。不同卡牌根据所在位置设置横向间距、纵向偏移和旋转角度，形成扇形布局；鼠标悬停时，卡牌会上移并放大

动画主要使用：

- `QPropertyAnimation`：实现位置、大小和透明度变化
- `QTimer::singleShot()`：延迟敌人行动和效果结算
- `QPainter`：绘制卡牌、状态和战斗特效
- 临时 `QLabel` 或 `QWidget`：显示伤害数字、闪光和投射物

动画结束后，临时控件通过 `deleteLater()` 回收，降低对象生命周期管理出错的风险

3.6 地图生成与路径控制

地图节点由 `MapRoomData` 表示：

```
1  struct MapRoomData
2  {
3      int id;
4      int layer;
5      int index;
6      MapNodeType type;
7      QList<int> nextRoomIds;
8      bool available;
9      bool completed;
10     bool pending;
11 };
```

`MapWidget` 先生成各层节点，再创建相邻层之间的有向连接。每个节点通过 `nextRoomIds` 保存可前往的下一层节点

节点具有三种主要状态：

- `available`：当前可选择
- `pending`：已经进入但尚未完成
- `completed`：已经完成

完成节点后，程序根据上一层已完成节点的连接关系更新下一层的可用状态

地图背景和连线由自定义画布在 `paintEvent()` 中使用 `QPainter` 绘制，节点按钮负责交互。这样可以分别处理视觉绘制和鼠标点击

地图结构支持 JSON 序列化。读取存档时，程序还会检查连接是否有效；若发现某个非终点节点没有后继，会重新生成连接，避免玩家因异常存档无法继续

3.7 数据驱动事件与其他功能页面

事件系统由 `RandomEventData`、`RandomEventChoice` 和 `RandomEventEffect` 组成。事件数据保存故事、图片和选项，效果数据保存效果类型、数值和目标 ID

一个选项可以组合多个标准效果，例如同时修改数值、添加卡牌或给予下一场战斗增益。`EventWidget` 只负责逐页展示故事和创建选项按钮，具体效果由事件逻辑和 `GameState` 处理

新增普通事件时，只需要：

1. 创建事件数据
2. 填写故事和选项
3. 配置效果列表
4. 将事件加入事件池

商店、奖励和宿舍页面也采用类似结构：界面负责显示和选择，数据修改通过 `GameState` 的统一接口完成。按钮常使用 Lambda 表达式捕获对应卡牌或下标，选择完成后再调用主窗口回调

3.8 JSON 存档、音频与资源管理

存档由 `MainWindow::saveRunToDisk()` 和 `loadRunFromDisk()` 组织，`GameState` 与 `MapWidget` 分别负责自身数据的序列化。文件主要包含版本号、游戏状态和地图状态：

```
1 {
2   "version": 1,
3   "state": {},
4   "map": {}
5 }
```

程序使用 `QSaveFile` 写入 JSON。它会先写入临时文件，提交成功后再替换正式存档，可以降低异常退出时破坏原文件的风险。读档时会检查 JSON 格式、版本号和必要字段

`AudioManager` 封装 `QMediaPlayer` 和 `QAudioOutput`，实现背景音乐播放、循环、暂停和音量控制。主窗口分别维护世界音乐和战斗音乐播放器，在页面切换时控制对应音乐

为了适配 Qt Creator 构建目录和发布目录，程序会从可执行文件目录、当前工作目录和源码目录中搜索 `resources` 文件夹，提高图片和音频在不同运行环境中的可用性

四、项目特色

4.1 鲜明的校园主题

本项目最大的特色是将卡牌爬塔玩法与校园生活结合。游戏中的敌人、事件、奖励和数值命名都围绕大学生生活展开，使玩家能够快速理解并产生共鸣

例如，“心情”对应生命值，“余额”对应金币，“平时分”和“期末成绩”对应游戏评分。这些设定让原本抽象的游戏数值变得更贴近学生生活

4.2 完整的单局游戏体验

游戏已经具备从开始到结束的完整闭环。玩家可以开始一局新游戏，在地图中选择路线，经历多种节点，逐步成长，最终挑战 Boss 并进入结算页面

相比只展示单个功能的演示程序，本项目更接近一个完整小游戏

4.3 策略性与随机性结合

游戏既有随机事件、随机奖励和随机敌人带来的不确定性，也有路线选择、卡牌选择、商店购买和战斗出牌带来的策略性

玩家每局都会面对不同局面，需要根据当前资源做出选择。这种设计提升了重复游玩的价值

4.4 内容丰富

项目包含多种卡牌、敌人、事件、遗物、药水和场景资源。除了核心战斗外，游戏还加入了剧情事件、商店、休息、奖励和结算系统，使体验更丰富

4.5 视听表现较完整

项目配有主菜单、地图、战斗、事件、卡牌、敌人等图片资源，也加入了背景音乐和战斗音乐。相比纯文字或简单按钮界面，游戏更有表现力和沉浸感

五、小组成员分工情况

孙靖：

- 地图界面、节点与路径系统设计
- 卡牌动画效果设计
- 数值调整与设计
- 路演视频录制与剪辑

吴迪：

- 战斗界面、战斗系统、随机事件设计
- 卡牌设计与敌人设计
- 数值调整与设计
- 路演视频录制

六、项目总结与反思

- 本项目灵感来源于天天爬塔的舍友。一开始我们认为复刻杀尖是一项大工程，也产生过畏难的心理，纠结到底要不要换一个主题，但最终我们还是下定决心做这个北大风格的杀尖。从结果来看，虽然项目还有一些不尽如人意的地方，但我们基本实现了一开始的设想，取得了令我们非常满意的效果
- 在开发的过程中，我们遇到了很多困难与挑战。如何设计出丝滑的动画效果？如何处理好 gui 操作与状态系统的同步？如何维护好游戏内的各种数据？如何进行高效的调试？如何同步好两人的代码、合作开发？这些复杂的问题和巨大的工作量耗费了我们大量的时间与精力，但在我们的不懈努力与 AI 的辅助下，我们还是顺利地完成了任务
- 在这次大作业中，我们进一步熟悉了较大规模 Qt 程序的开发流程，也加深了对面向对象设计、模块协作和界面状态管理的理解。相比平时较为独立的编程作业，本项目需要同时考虑一个复杂有机的系统，对我们来说是一个不小的挑战，但同时也是宝贵的学习与锻炼的机会
- 当然，在本次作业过程中，我们发现我们还是存在不擅长团队协作共同开发、写出的代码不易维护等问题。总而言之，这次大作业为初次合作的我们积累了宝贵的协作经验，使我们对完整软件项目的开发过程有了更实际的认识，是一次难能可贵的学习经历